is used to predict the next token $t_{|T|+1}$ (Figure 3). Thus, we simplify $\boldsymbol{A}$ by focusing on the attentions on the last input token, denoted as $\boldsymbol{A}_{\text{simp}}$. This reduces the size of attentions to $L \times H \times |T|$. Findings in previous works support that $\boldsymbol{A}_{\text{simp}}$ is sufficient to reveal the focus of the prediction token (Brunner et al., 2020; Geva et al., 2023). In our experimental setup, we also find that analysis on $\boldsymbol{A}_{\text{simp}}$ gives similar results to that on $\boldsymbol{A}$ (Appendix B.4).

Attention head pooling. Many existing attention-based analysis methods use pooling (e.g., mean pooling or max pooling) on attention heads for simplicity (Abnar and Zuidema, 2020; Manning et al., 2020; Murty et al., 2023). We follow this idea and take the mean value across all attention heads for our analysis. Then, the size of $\boldsymbol{A}_{\text{simp}}$ is further reduced to $L \times |T|$.

Ignoring attention weights within the statement (LLaMA). For LLaMA on the two natural language reasoning tasks, a statement S_i could contain multiple tokens, i.e., $|T| >> |\mathcal{S}|$. Thus, the size of $\boldsymbol{A}_{\text{simp}}$ can still be large. To further simplify $\boldsymbol{A}_{\text{simp}}$ under this setting, we regard all tokens of a statement as a hypernode. That is, we ignore attentions within hypernodes and focus on attentions across hypernodes. As shown in Figure 4, we can get $\boldsymbol{A}_{\text{simp}}^{\text{cross}}$ via *mean pooling* on all tokens of a statement and *max pooling* on all tokens of Q as:[5]

$$\boldsymbol{A}_{\text{simp}}^{\text{cross}}(l,h)[i] = \max_{t_{j'} \in Q} \left(\underset{t_{i'} \in S_i}{\text{mean}} \left(\boldsymbol{A}(l,h)[i',j'] \right) \right).$$

The size of simplified cross-hypernode attention matrix $\boldsymbol{A}_{\text{simp}}^{\text{cross}}$ is further reduced to $L \times (|\mathcal{S}| + 1)$.

Pruning layers (LLaMA). Large LMs (e.g., LLaMA) are very deep (i.e., L is large), and are pretrained for a large number of tasks. Thus, they have many redundant parameters for performing the reasoning task. Inspired by Rogers et al. (2020), we prune the useless layers of LLaMA for the reasoning task and probe attentions of the remaining layers. Specifically, we keep a minimum number of layers that maintain the LM's performance on a held-out development set, and deploy our analysis on attentions of these layers. For 4-shot LLaMA, 13/15 (out of 32) layers are removed for ProofWriter and ARC respectively. For finetuned LLaMA, 18 (out of 32) layers are removed for ProofWriter. More details about the attention pruning can be found in Appendix C.2.

[5]We take mean pooling for statements since max pooling cannot differentiate statements with many overlapped words. We take max pooling for Q since it is more differentiable for the long input text. In practice, users can select different pooling strategies based on their own requirements.

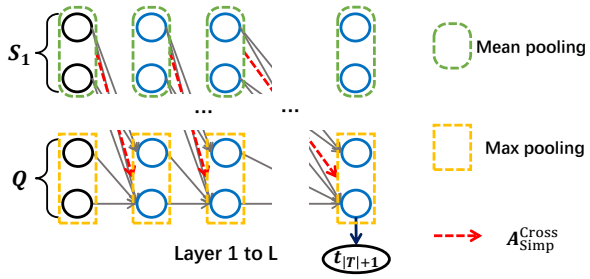


Figure 4: An illustration of the way to extend our attention simplification method. In this example, S_1 and Q are both composed of 2 tokens. Thus, there are 2×2 attentions. We mean pool all the tokens within the statements (remaining 1×2 attentions), and max pool all the tokens in the question (remaining 1×1 attention).

3.3 Simplification of the Probing Task

We simplify the problem of predicting G by breaking it down into two classification problems: classifying if the statement is useful or not, and classifying the height of the statement in the tree.

$$P(G|\boldsymbol{A}_{\text{simp}}) = P(V|\boldsymbol{A}_{\text{simp}}) \cdot P(G|V, \boldsymbol{A}_{\text{simp}}).$$

Here, V is the set of nodes in G. $P(V|\boldsymbol{A}_{\text{simp}})$ (binary classification) measures if LMs can select useful statements from the input based on attentions, revealing if LMs correctly focus on useful statements. Given the set of nodes in G, the second probing task is to decide the reasoning tree. We model $P(G|V, \boldsymbol{A}_{\text{simp}})$ as the multiclass classification for predicting the height of each node. For example, when the reasoning tree depth is 2, the height label set is $\{0, 1, 2\}$. Note that for the three reasoning tasks considered by us, G is always a simple tree that has multiple leaf nodes but one intermediate node at each height.[6]

3.4 Probing Score

To limit the amount of information the probe learns about the probing task, we use a non-parametric classifier: k-nearest neighbors (kNN) to perform the two classification tasks. We use $S_{\text{F1}}(V|\boldsymbol{A}_{\text{simp}})$ and $S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{simp}})$ to denote their F1-Macro scores. To better interpret the probing results, we

[6]In practice, many reasoning graphs can be formalized as the chain-like tree structure (Wei et al., 2022). We leave the analysis with more complex G for future work.

introduce a random probe baseline as the control task (Hewitt and Liang, 2019) and instead of looking at absolute probing values, we interpret the probing scores compared to the score of the random baseline. The probe scores are defined as:

$$S_{\text{P1}} = \frac{S_{\text{F1}}(V|\boldsymbol{A}_{\text{simp}}) - S_{\text{F1}}(V|\boldsymbol{A}_{\text{rand}})}{1 - S_{\text{F1}}(V|\boldsymbol{A}_{\text{rand}})}, \quad (1)$$

$$S_{\text{P2}} = \frac{S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{simp}}) - S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{rand}})}{1 - S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{rand}})}, \quad (2)$$

where $\boldsymbol{A}_{\text{rand}}$ is the simplified attention matrix given by a randomly initialized LM. After normalization, we have the range of our probing scores: $S_{\text{P1}}, S_{\text{P2}} \in [0, 1]$. Small values mean that there is no useful information about G in attention, and large values mean that the attention patterns indeed contain much information about G.

4 Mechanistic Probing of LMs

We use the probe to analyze how LMs perform the reasoning tasks. We first verify the usefulness of attentions in understanding the reasoning process of LMs by visualizing $\boldsymbol{A}_{\text{simp}}$ (§4.1). Then, we use the probe to quantify the information of G contained in $\boldsymbol{A}_{\text{simp}}$ (§4.2). Finally, we report layer-wise probing results to understand if LMs are reasoning procedurally across their architecture (§4.3). [7]

4.1 Attention Visualization

We first analyze $\boldsymbol{A}_{\text{simp}}$ on the k-th smallest element task via visualizations. We permute $\boldsymbol{A}_{\text{simp}}$ arranging the numbers in ascending order and denote this permulation as $\pi(\boldsymbol{A}_{\text{simp}})$. We show visualizations of $\mathbb{E}[\pi(\boldsymbol{A}_{\text{simp}})]$ on the test data in Figure 5. We observe that when GPT-2 tries to find the k-th smallest number, the prediction token first focuses on top-k numbers in the list with bottom layers. Then, the correct answer is found in the top layers. These findings suggest that GPT-2 solves the reasoning task in two steps following the reasoning tree G. We further provide empirical evidence in Appendix B.4 to show that analysis on $\boldsymbol{A}_{\text{simp}}$ gives similar conclusions compared to that on $\boldsymbol{A}$.

4.2 Probing Scores

Next, we use our `MechanisticProbe` to quantify the information of G contained in $\boldsymbol{A}_{\text{simp}}$ (GPT-2) or $\boldsymbol{A}^{\text{cross}}_{\text{simp}}$ (LLaMA).

GPT-2 on k-th smallest element ($\boldsymbol{A}_{\text{simp}}$). We consider two versions of GPT-2: a pretrained version and a finetuned version (GPT-2$_{\text{FT}}$). We report our two probing scores (Eq. 1 and Eq. 2) with different k in Table 1. The unnormalized F1-macro scores (i.e., $S_{\text{F1}}(V|\boldsymbol{A}_{\text{simp}})$ and $S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{simp}})$) can be found in Appendix B.2.

Table 1: Probing scores for GPT-2 models on synthetic reasoning tasks with different k. We also provide the test accuracy of these finetuned models for reference. Note that when $k = 1$, the depth of G is 0, and $S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{simp}})$ is always equal to 1. Thus, we leave these results blank. Results show that we can clearly detect G from attentions of GPT-2$_{\text{FT}}$.

k	Test Acc.		S_{P1}		S_{P2}	
	GPT-2	GPT-2$_{\text{FT}}$	GPT-2	GPT-2$_{\text{FT}}$	GPT-2	GPT-2$_{\text{FT}}$
1	0.00	99.63	7.09	92.94	-	-
2		99.42	5.88	93.71	20.65	98.05
3		98.23	13.48	91.62	13.59	95.76
4		94.89	20.73	88.34	8.04	92.52
5		93.38	24.15	87.54	13.81	88.17
6		92.62	24.82	86.89	11.18	95.06
7		91.37	25.87	76.61	< 1	91.56
8		91.29	22.30	78.73	1.06	93.93

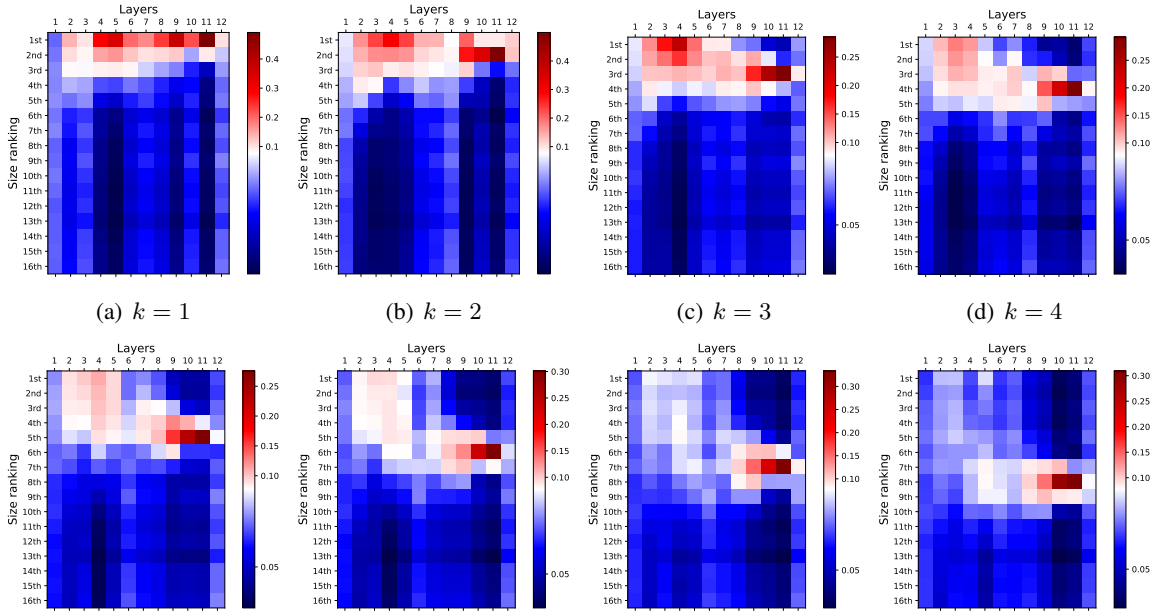
These results show that without finetuning, GPT-2 is incapable of solving the reasoning task, and we can detect little information about G from GPT-2's attentions.[8] However, for GPT-2$_{\text{FT}}$, which has high test accuracy on the reasoning task, `MechanisticProbe` can easily recover the reasoning tree G from $\boldsymbol{A}_{\text{simp}}$. This further confirms that GPT-2$_{\text{FT}}$ solves this synthetic reasoning task following G in Figure 2 (left).

LLaMA on ProofWriter and ARC ($\boldsymbol{A}^{\text{cross}}_{\text{simp}}$). Similarly, we use `MechanisticProbe` to probe LLaMA on the two natural language reasoning tasks. For efficiency, we randomly sampled 1024 examples from the test sets for our analysis. When depth = 0, LLaMA only needs to find out the useful statements for reasoning (S_{P1}). When depth=1, LLaMA needs to determine the next reasoning step.

Probing results with different numbers of input statements (i.e., $|\mathcal{S}|$) are in Table 2. Unnormalized classification scores can be found in Appendix C.5. It can be observed that all the probing scores are much larger than 0, meaning that the attentions indeed contain information about G.

[7] We have additional empirical explorations in the Appendix. Appendix B.6 shows that different finetuning methods would not influence probing results. Appendix B.7 explores how the reasoning task difficulty and LM capacity influence the LM performance. Appendix B.8 researches the relationships between GPT-2 performance and our two probing scores.

[8] Visualization of $\mathbb{E}[\pi(\boldsymbol{A}_{\text{simp}})]$ for (pretrained) GPT-2 can be found in Appendix B.3. It shows that GPT-2 can somehow find out the largest number from the number list.



(a) $k = 1$ (b) $k = 2$ (c) $k = 3$ (d) $k = 4$

(e) $k = 5$ (f) $k = 6$ (g) $k = 7$ (h) $k = 8$

Figure 5: Visualization of $\mathbb{E}[\pi(\boldsymbol{A}_{\text{simp}})]$ for k ranging from 1 to 8. Note that $m = 16$ here, and when $k > 8$, the reasoning task becomes finding k-th largest number. The visualizations are roughly the same without loss of generality. The x-axis represents the layer index and the y-axis represents the size (rank) of the number. The value of each cube is the attention weight (better view in color). Results show that layers of finetuned GPT-2 have different functions: bottom layers focusing on top-k numbers and top layers focusing on k-smallest numbers.

Table 2: Probing results for LLaMA on natural language reasoning. Note that when depth $= 0$, $S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{simp}})$ is always equal to 1. When depth $= 1$ and $|\mathcal{S}| = 2$, all input statements are useful. $S_{\text{F1}}(V|\boldsymbol{A}_{\text{simp}})$ is always equal to 1. We leave these results blank. Regarding ARC, we only report the scores under the 4-shot setting. Results show that attentions of LLaMA contain some information about selecting useful statements for ProofWriter but much information for ARC. Regarding determining reasoning steps, attentions of LLaMA contain much information for both ProofWriter and ARC.

ProofWriter							
Depth	$\lvert\mathcal{S}\rvert$	Test Acc.		S_{P1}		S_{P2}	
		LLaMA	LLaMA_{FT}	LLaMA	LLaMA_{FT}	LLaMA	LLaMA_{FT}
0	All	81.72	100	57.21	49.08	-	-
1	2	94.81		-	-	100	100
	4	95.12		44.83	48.14	93.34	96.22
	8	92.19		27.39	40.09	83.75	96.44
	12	90.53		26.23	32.70	77.58	93.45
	16	89.55		17.18	21.07	77.85	89.31
	20	88.38		11.10	15.84	79.99	94.11
	24	86.13		9.39	17.33	80.32	94.42
ARC							
1	-	56.32	-	97.49	-	61.73	-
2	-	55.41		96.49	-	53.40	-

Looking at S_{P1} on ProofWriter, when the number of input statements is small, `MechanisticProbe` can clearly decide the useful statements based on attentions. However, it becomes harder when there are more useless statements (i.e., $|S|$ is large). However, for ARC, our probe can always detect useful statements from attentions easily.

Looking at S_{P2}, we notice that our probe can easily determine the height of useful statements based on attentions on both ProofWriter and ARC datasets. By comparing the probing scores on ProofWriter, we find that LLaMA_{FT} always has higher probing scores than 4-shot LLaMA, implying that finetuning with supervised signals makes the LM to follow the reasoning tree G more clearly. We also notice that 4-shot LLaMA is affected more by the number of useless statements than LLaMA_{FT}, indicating a lack of robustness of reasoning in the few-shot setting.

4.3 Layer-wise Probing

After showing that LMs perform reasoning following oracle reasoning trees, we investigate how this reasoning happens inside the LM layer-by-layer.

GPT-2 on k-th smallest element. In order to use our probe layer-by-layer, we define the set of simplified attentions before layer l as $\boldsymbol{A}_{\text{simp}}(:l) = \{\boldsymbol{A}_{\text{simp}}(l')|l' \leq l\}$. Then, we report our probing scores $S_{\text{P1}}(l)$ and $S_{\text{P2}}(l)$ on these partial attentions from layer 1 to layer 12. We denote these layer-wise probing scores as $S_{\text{F1}}(V|\boldsymbol{A}_{\text{simp}}(:\ l))$ and $S_{\text{F1}}(G|V, \boldsymbol{A}_{\text{simp}}(:l))$.